

Módulo 0 — Fundamentos (Base Conceptual)

Curso: Arquitectura de Software Profesional con Clean Architecture, DDD y SOLID

Caso de uso guía: **CU-CLI-001 — Alta de Cliente**

Objetivo del módulo

Construir el modelo mental correcto **antes de escribir código**. Comprender por qué la arquitectura importa, qué problemas resuelve y cuáles son los pilares conceptuales sobre los que se construye todo el sistema.

Este módulo es completamente conceptual. No se escribe código todavía. El objetivo es que, al finalizar, el alumno pueda *pensar* en términos de arquitectura antes de abrir el editor.

Primero entender → Luego modelar → Luego integrar → Finalmente exponer

Temas del módulo

Tema 1	Introducción a la arquitectura de software
Tema 2	Problemas del desarrollo sin arquitectura
Tema 3	¿Qué es una dependencia?
Tema 4	¿Qué es Clean Architecture?
Tema 5	Capas y dependencias en CommerX
Tema 6	Principios SOLID (visión general)
Tema 7	Introducción a DDD
Tema 8	¿Qué es un modelo de dominio?

Tema 1 — Introducción a la arquitectura de software

1. ¿Qué es?

La **arquitectura de software** es el conjunto de decisiones de organización de alto nivel que definen cómo se divide y conecta un sistema. Determina qué partes existen, qué hace cada una y cómo se comunican entre sí.

No es un detalle técnico menor: es la columna vertebral del sistema. Una arquitectura bien pensada hace que el software sea *mantenible*, *escalable* y *comprensible*. Una mala arquitectura convierte al sistema en una deuda técnica que cada día cuesta más modificar.

2. Analogía

Imaginemos un **edificio de oficinas**. Antes de construirlo, un arquitecto decide cuántos pisos tendrá, qué función cumplirá cada uno y cómo se conectan entre sí (escaleras, ascensores, puertas). Nadie construye un edificio de veinte pisos sin un plano previo.

Edificio de oficinas – analogía con las capas del sistema:

Piso 4 → Recepción / Atención al cliente	← lo que el usuario ve (UI)
Piso 3 → Administración / Coordinación	← quien organiza el trabajo
Piso 2 → Operaciones / El trabajo real	← las reglas del negocio
Piso 1 → Infraestructura / Servicios	← luz, agua, internet, servidores

Cada piso tiene tareas bien definidas.
La recepción no repara las cañerías.
Las cañerías no atienden clientes.
Si se cambia el proveedor de internet (piso 1),
la recepción (piso 4) no se entera ni se modifica.

El software sin arquitectura es como un edificio donde todos hacen todo: el recepcionista arregla las cañerías, el plomero atiende clientes y nadie sabe cuál es su responsabilidad. Funciona con tres personas, pero con treinta colapsa.

3. Características esenciales

- **Separación de responsabilidades:** cada parte del sistema tiene un único propósito claro.
- **Independencia tecnológica:** las reglas del negocio no dependen de frameworks ni bases de datos.
- **Testabilidad:** cada componente puede verificarse de forma aislada, sin levantar todo el sistema.
- **Mantenibilidad:** los cambios en una parte no rompen el resto.
- **Comprensibilidad:** cualquier desarrollador nuevo puede entender el sistema en poco tiempo.

4. ¿Por qué se usa?

Los sistemas de software crecen con el tiempo. Una funcionalidad que hoy tarda un día en implementarse puede tardar semanas si el código está desorganizado. La arquitectura es una **inversión inicial** que genera retorno a medida que el sistema escala.

Sin arquitectura	Cambios costosos, errores frecuentes, código difícil de leer, tests imposibles de escribir.
Con arquitectura	Cambios predecibles, errores localizados, código comprensible, tests sencillos y confiables.

Tema 2 — Problemas del desarrollo sin arquitectura

1. ¿Qué es el "código espagueti"?

Se llama **código espagueti** a aquel donde la lógica de negocio, el acceso a datos, la validación y la presentación están mezclados sin ningún criterio de organización. El flujo del programa es tan enredado como un plato de espaguetis: imposible de seguir y aún más difícil de modificar.

2. Analogía

Imaginar una cocina donde el mismo cocinero también cobra la cuenta, atiende las mesas, lava los platos y hace la contabilidad al mismo tiempo. Con tres clientes puede funcionar. Con treinta, el sistema colapsa. En software: un método que valida, guarda en la base de datos y envía un email tiene exactamente el mismo problema.

3. Errores comunes en proyectos sin arquitectura

- Lógica de negocio directamente en el controlador o en el formulario de pantalla.
- Consultas SQL mezcladas con la lógica de presentación.
- Objetos con decenas de propiedades y sin comportamiento (entidades sin vida propia).
- Las mismas validaciones repetidas en múltiples partes del sistema.
- Clases con nombres como `Manager`, `Helper` o `Utils` que hacen de todo.
- Imposibilidad de escribir tests porque todo está mezclado y acoplado.

4. Qué NO hacer — Ejemplo de código sin arquitectura

El siguiente ejemplo muestra cómo **NO** debe estructurarse un sistema. Toda la responsabilidad del alta de cliente cae sobre un único método:

```
1 // ✗ MAL: toda la lógica mezclada en un solo método del controlador
2 public IActionResult CreateClient(string name, string email, string document, DateTime
  birthDate)
3 {
4     // ✗ regla de negocio mezclada con la pantalla: ¿dónde está "mayor de edad"?
5     if (string.IsNullOrEmpty(name)) return BadRequest("Nombre requerido");
6     if (!email.Contains("@")) return BadRequest("Email inválido");
7     // ✗ la validación de edad es incorrecta: ignora mes y día
8     if ((DateTime.Now.Year - birthDate.Year) < 18) return BadRequest("Menor de edad");
9
10    // ✗ acceso directo a la base de datos: vulnerable a inyección SQL
11    var sql = $"INSERT INTO Clients VALUES ('{name}', '{email}', '{document}')";
```

```
12     _connection.Execute(sql);  
13  
14     return Ok("Cliente creado");  
15 }
```

⚠ Problemas identificados:

- La validación de mayoría de edad es **incorrecta**: restar años no alcanza para saber si alguien cumplió 18.
- La regla de negocio vive en el controlador: si hay otro formulario, hay que copiarla y puede desincronizarse.
- SQL concatenado directamente: si `name` contiene comillas, la consulta se rompe o se explota.
- Imposible de testear: para probar este método hay que levantar un servidor HTTP y una base de datos real.
- Si la regla cambia a "mayor de 21 años", hay que buscarla en todo el código fuente.

Tema 3 — ¿Qué es una dependencia?

Antes de hablar de arquitectura en capas, es necesario entender un concepto fundamental: la **dependencia**. Este concepto es la base de todo lo que viene a continuación.

1. ¿Qué es una dependencia en software?

Una clase **depende** de otra cuando la necesita para funcionar. Es decir, si la clase A usa a la clase B, entonces A *depende* de B. Si B cambia o desaparece, A se ve afectada.

Dicho de otra forma: **si A no puede compilar o funcionar sin B, entonces A depende de B.**

2. Analogía del mundo real

Imaginar un **televisor** enchufado a la corriente eléctrica. El televisor *depende* de la red eléctrica para funcionar. Si la red eléctrica cambia de 220V a 110V, el televisor se daña o deja de funcionar. El televisor está *acoplado* a una tecnología específica de infraestructura.

Ahora imaginar que el televisor tiene un adaptador de corriente universal. El televisor ya no depende directamente de la red: depende del adaptador. Podemos cambiar el adaptador para 110V o 220V sin tocar el televisor. Esta es exactamente la idea detrás de *invertir las dependencias* en software.

3. Dependencia en código — ejemplo sencillo

El siguiente ejemplo muestra qué significa que una clase dependa de otra. Por ahora el código es simple e ilustrativo:

```
1 // Clase SqlClientRepository: guarda clientes en SQL Server
2 public class SqlClientRepository
3 {
4     public void Save(string clientName)
5     {
6         // guarda en SQL Server...
7     }
8 }
9
10 // ✗ MAL: CreateClientUseCase depende DIRECTAMENTE de SqlClientRepository
11 // Si mañana queremos usar PostgreSQL, hay que modificar CreateClientUseCase
```

```
12 public class CreateClientUseCase
13 {
14     // el caso de uso "conoce" y depende de la clase concreta SqlClientRepository
15     private SqlClientRepository _repository = new SqlClientRepository();
16
17     public void Execute(string clientName)
18     {
19         _repository.Save(clientName); // acoplado a SQL Server
20     }
21 }
```

🔍 ¿Qué problema genera esta dependencia directa?

- `CreateClientUseCase` está *atado* a SQL Server. Si cambia la base de datos, hay que modificar el caso de uso.
- Para testear `CreateClientUseCase` en aislamiento, necesitamos una base de datos SQL Server real funcionando.
- El negocio (caso de uso) conoce un detalle de infraestructura (SQL Server). Eso viola la separación de responsabilidades.

4. La solución: depender de abstracciones

En lugar de que `CreateClientUseCase` dependa de `SqlClientRepository`, puede depender de un **contrato** que diga *"necesito algo que pueda guardar clientes"*, sin importar cómo lo hace ni con qué base de datos.

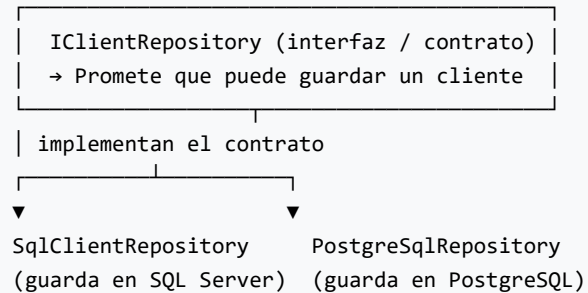
Ese contrato, en C#, se llama **interfaz**.

¿Qué es una interfaz en C#?

Una interfaz es un contrato que define QUÉ puede hacer algo, sin especificar CÓMO lo hace.

Es como el enchufe de la pared: define la forma y el voltaje, pero no importa si la electricidad viene de una central hidroeléctrica o de un panel solar. Cualquier cosa que "encaje" en ese contrato funciona.

Ejemplo conceptual:



```
1 // ✅ BIEN: CreateClientUseCase depende de la INTERFAZ (contrato), no de SQL Server
2 // Puede funcionar con cualquier clase que cumpla el contrato IClientRepository
3 public class CreateClientUseCase
4 {
5     // depende de la interfaz, no de una clase concreta
6     private readonly IClientRepository _repository;
7
8     // recibe la implementación desde afuera (quién la provea no importa)
9     public CreateClientUseCase(IClientRepository repository)
10    {
11        _repository = repository;
12    }
13
14    public void Execute(string clientName)
15    {
16        _repository.Save(clientName); // no sabe ni le importa con qué base de datos
17    }
18 }
```

✅ Ventajas de depender de la abstracción:

- Se puede cambiar de SQL Server a PostgreSQL sin tocar `CreateClientUseCase`.
- Para testear, se puede pasar una implementación "falsa" que no use ninguna base de datos real.
- El caso de uso no conoce detalles de infraestructura: cumple el principio de separación de responsabilidades.

Este concepto de *"depender de abstracciones"* es la base de toda Clean Architecture. Todo lo que veremos de aquí en adelante se construye sobre él.

Tema 4 — ¿Qué es Clean Architecture?

1. ¿Qué es?

Clean Architecture es un modelo de organización del software propuesto por Robert C. Martin (conocido como "Uncle Bob") que organiza el sistema en **capas concéntricas**, como los anillos de una cebolla.

La regla más importante se llama la **Regla de Dependencia**:

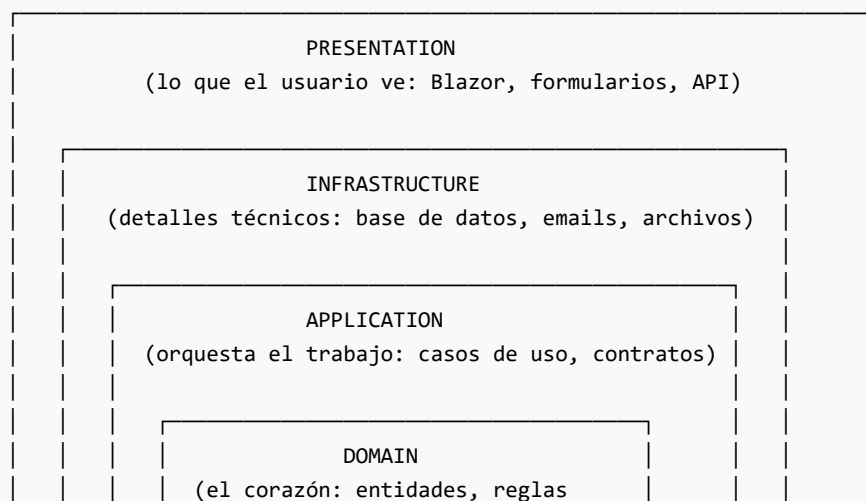
Las dependencias del código fuente SÓLO pueden apuntar hacia adentro.
La capa exterior puede conocer a la interior.
La capa interior NUNCA puede conocer a la exterior.

Esto garantiza que el núcleo del sistema, que contiene las reglas del negocio, no dependa de ninguna tecnología externa. Si cambia la base de datos, el framework o la interfaz de usuario, el núcleo no se modifica.

2. Analogía

Pensar en una **cebolla**. El centro no sabe que hay capas que lo envuelven. Si pelamos la capa exterior, el centro sigue igual. En software: si reemplazamos la base de datos (capa exterior), el dominio (centro) no se entera.

3. Las cuatro capas del sistema



de negocio, objetos de valor)

- Presentation puede usar Application.
- Infrastructure puede usar Application y Domain.
- Application puede usar Domain.
- Domain NO usa nadie: es completamente autónomo.

4. ¿Qué hace cada capa? — Explicación simple

Domain	Es el corazón del sistema . Contiene las entidades del negocio (como Client) y todas las reglas que deben cumplirse (mayor de edad, email válido, etc.). No conoce bases de datos, pantallas ni ninguna tecnología.
Application	Coordina el trabajo . Contiene los casos de uso (como "alta de cliente") que orquestan el dominio. Define los contratos (interfaces) que necesita, pero no los implementa. Depende del Domain, pero no de Infrastructure ni de Presentation.
Infrastructure	Implementa los detalles técnicos . Aquí vive el acceso a la base de datos (Entity Framework), el envío de emails, la lectura de archivos, etc. Implementa los contratos definidos por Application.
Presentation	Es lo que el usuario ve y toca . Contiene los formularios, los controladores de la API y las pantallas de Blazor. Recibe datos del usuario y los entrega a Application.

5. ¿Por qué se usa?

- **Independencia de la base de datos:** migrar de SQL Server a PostgreSQL es un cambio solo en Infrastructure.
- **Independencia del framework:** cambiar de ASP.NET a gRPC no afecta el dominio.
- **Testabilidad total:** el dominio puede testearse sin ninguna dependencia externa.
- **Independencia de la UI:** la lógica de negocio existe y funciona antes de que exista la pantalla.

Tema 5 — Capas y dependencias en el proyecto CommerX

1. Estructura de proyectos

En la solución **CommerX**, cada capa es un proyecto independiente de C#. Esta separación física garantiza que las dependencias sean explícitas y verificadas por el propio compilador: si intentamos usar EF Core desde el dominio, el proyecto directamente **no compila**.

```

CommerX.sln
+---src
+---CommerX.Domain                ← Corazón: entidades, VOs, excepciones
|   +---Common
|   |   +---Entities              ← Clase base Entity<TId>
|   |   +---ValueObjects          ← Clase base ValueObject
|   +---Customers
|       +---Entities              ← Client.cs
|       +---ValueObjects          ← Email.cs, Document.cs, BirthDate.cs, etc.
|       +---Exceptions            ← InvalidEmailException.cs, etc.
|
+---CommerX.Application           ← Coordinador: casos de uso, DTOs, contratos
|   +---Customers
|       +---UseCases              ← CreateClientUseCase.cs
|       +---DTOs                 ← CreateClientRequest.cs, ClientResponse.cs
|       +---Interfaces            ← IClientRepository.cs (contrato, no implementación)
|
+---CommerX.Infrastructure        ← Detalles técnicos: EF Core, repositorios concretos
|   +---Persistence
|       +---Configurations        ← ClientEntityConfiguration.cs
|       +---Repositories          ← ClientRepository.cs (implementa IClientRepository)
|
+---CommerX.InterfaceAdapter     ← Lo que el usuario ve: API, Blazor ViewModels
+---Customers
+---Controllers                  ← ClientsController.cs
+---ViewModels                   ← CreateClientViewModel.cs
  
```

2. Regla de dependencias aplicada al proyecto

CommerX.Domain	No referencia ningún otro proyecto de la solución. Es completamente autónomo. No conoce EF Core, no conoce la API.
CommerX.Application	Referencia únicamente a Domain . Define contratos (interfaces) como IClientRepository , pero no los implementa.
CommerX.Infrastructure	Referencia a Application y Domain . Implementa los contratos: ClientRepository cumple el contrato IClientRepository usando EF Core.
CommerX.InterfaceAdapter	Referencia a Application . Recibe datos del usuario y llama a los casos de uso. No accede directamente al dominio ni a la base de datos.

3. Flujo completo de CU-CLI-001 — Alta de Cliente

Para visualizar cómo fluye la información a través de las capas, veamos el recorrido completo desde que el usuario completa el formulario hasta que el cliente queda guardado en la base de datos:

```
[1] PRESENTATION
El usuario completa el formulario en Blazor
→ Se crea un objeto CreateClientRequest con los datos ingresados

[2] APPLICATION
CreateClientUseCase recibe el CreateClientRequest
→ Ordena al dominio que cree el cliente con esos datos

[3] DOMAIN
Client.Create(...) valida cada dato usando Value Objects:
→ Email.Create(...) valida el formato del email
→ Document.Create(...) valida el formato del documento
→ BirthDate.Create(...) valida que sea mayor de edad
Si algo es inválido, se lanza una excepción de dominio

[4] APPLICATION (continúa)
Si el cliente se creó sin errores,
→ IClientRepository.SaveAsync(client) guarda el cliente
(Application pide guardar; Infrastructure lo hace)

[5] INFRASTRUCTURE
ClientRepository implementa el contrato IClientRepository
→ Usa EF Core para persistir el cliente en SQL Server

[6] PRESENTATION
```



El caso de uso termina y devuelve una respuesta
→ La pantalla muestra "Cliente registrado correctamente"

Tema 6 — Principios SOLID (visión general)

1. ¿Qué son los principios SOLID?

SOLID es un acrónimo que agrupa cinco principios de diseño orientado a objetos, formulados por Robert C. Martin. Son guías de *buen diseño* que, aplicadas correctamente, producen código flexible, mantenible y comprensible.

No son reglas rígidas que siempre se aplican de la misma manera: son principios que hay que entender e interpretar según el contexto. A lo largo del curso los aplicaremos uno a uno, de forma progresiva.

2. Los cinco principios

S — Single Responsibility Principle (SRP) — Principio de Responsabilidad Única

"Una clase debe tener una, y solo una, razón para cambiar."

Cada clase debe hacer **una sola cosa** y hacerla bien. Si una clase tiene que cambiar por dos motivos distintos, probablemente está haciendo demasiado.

✓ Correcto	<code>Email</code> solo valida emails. <code>Document</code> solo valida documentos. <code>CreateClientUseCase</code> solo orquesta el alta de un cliente.
✗ Incorrecto	Una clase <code>ClientManager</code> que valida el email, guarda en la base de datos y envía un correo de bienvenida.

O — Open/Closed Principle (OCP) — Principio Abierto/Cerrado

"El software debe estar abierto para extensión, pero cerrado para modificación."

Agregar nueva funcionalidad no debería requerir modificar el código que ya funciona. Se extiende creando nuevas clases o implementaciones, no editando las existentes.

Ejemplo en CommerX	Si se agrega un nuevo tipo de documento (pasaporte internacional), no se modifica la clase <code>Document</code> existente: se extiende el sistema con nueva lógica.
--------------------	--

L — Liskov Substitution Principle (LSP) — Principio de Sustitución de Liskov

"Cualquier lugar que use un tipo base debe poder usar un subtipo sin que nada se rompa."

Si en cualquier lugar del código se usa el contrato `IClientRepository`, debe funcionar igual sin importar si la implementación concreta usa SQL Server, PostgreSQL o incluso una lista en memoria para tests.

I — Interface Segregation Principle (ISP) — Principio de Segregación de Interfaces

"Una clase no debería verse obligada a implementar métodos que no necesita."

Los contratos (interfaces) deben ser específicos y pequeños. En CommerX, `IClientRepository` solo expone operaciones relacionadas con clientes. No mezcla operaciones de otras entidades del sistema.

D — Dependency Inversion Principle (DIP) — Principio de Inversión de Dependencias

"Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones (contratos / interfaces)."

Este es exactamente el concepto explicado en el Tema 3. `CreateClientUseCase` (nivel alto: lógica de negocio) no debe depender de `SqlClientRepository` (nivel bajo: detalle técnico). Ambos deben depender del contrato `IClientRepository`.

```
SIN DIP: CreateClientUseCase → SqlClientRepository (acoplamiento directo)
CON DIP: CreateClientUseCase → IClientRepository ← SqlClientRepository
(ambos dependen de la abstracción, no uno del otro)
```

3. SOLID en CommerX — tabla de referencia

SRP	Cada Value Object valida solo su propio concepto. Cada caso de uso hace una sola cosa.
OCP	Nuevas validaciones se agregan con nuevas clases, no modificando las existentes.
LSP	Cualquier repositorio concreto puede reemplazar a otro mientras cumpla el contrato.
ISP	Contratos específicos por módulo. No existe una interfaz "dios" con todo.
DIP	Application depende de contratos. Infrastructure los implementa. Nunca al revés.

Tema 7 — Introducción a Domain-Driven Design (DDD)

1. ¿Qué es DDD?

Domain-Driven Design (Diseño guiado por el dominio) es un enfoque de desarrollo propuesto por Eric Evans en su libro homónimo de 2003. Su idea central es simple pero poderosa:

El código debe reflejar el lenguaje y los conceptos del negocio real,
no los tecnicismos de la base de datos ni del framework.

El **dominio** es el área de conocimiento del negocio que el sistema modela. En CommerX, el dominio es la gestión de clientes comerciales. DDD dice que ese conocimiento debe vivir en el código de forma explícita y reconocible por cualquier persona del negocio.

2. Analogía

Un contador le dice al equipo de desarrollo: *"Un cliente debe ser mayor de edad para operar en el sistema."*

Con DDD, esa regla se traduce a código que el propio contador podría leer y reconocer. La clase se llama `Client`, no `TblClientes`. La regla vive en el método `Client.Create()`, no en un stored procedure de base de datos ni en una condición enterrada en un controlador.

El código habla el mismo idioma que el negocio. A eso se le llama **Lenguaje Ubicuo** (*Ubiquitous Language*): un vocabulario compartido entre desarrolladores y expertos del negocio.

3. Conceptos clave de DDD utilizados en el curso

Entidad (Entity)	Objeto que tiene identidad propia. Existe en el tiempo y puede cambiar sus datos, pero sigue siendo el mismo objeto por su identificador. Ejemplo: un <code>Client</code> con Id = 5 es siempre el mismo cliente aunque cambie su teléfono.
Value Object (VO)	Objeto definido por sus valores, sin identidad propia. Es inmutable: una vez creado, no cambia. Si el valor cambia, se crea uno nuevo. Ejemplo: <code>Email("juan@mail.com")</code> es igual a <code>Email("juan@mail.com")</code> sin importar cuándo se crearon.
Aggregate Root	La entidad principal que controla el acceso a todo su grupo de objetos. Nadie modifica los Value Objects internos del cliente sin pasar por <code>Client</code>.
Repository (Repositorio)	Abstracción del acceso a datos. El dominio define el contrato (interfaz <code>IClientRepository</code>) sin saber cómo se implementa. Infraestructura provee la implementación concreta con EF Core.
Domain Exception	Una excepción que describe la violación de una regla de negocio. Ejemplo: <code>InvalidEmailException</code>, <code>InvalidAgeException</code>. Son excepciones propias del dominio, no del framework.
Ubiquitous Language	El vocabulario compartido entre el negocio y el desarrollo. Los nombres de las clases, métodos y propiedades reflejan los términos que usa el negocio, no siglas ni abreviaciones técnicas.

4. ¿Por qué se usa DDD?

- Las reglas de negocio son *explícitas* y viven en un único lugar: el dominio.
- El código modela el negocio real, no las tablas de la base de datos.
- El dominio es completamente testeable sin necesitar infraestructura.
- Reduce la brecha de comunicación entre el equipo técnico y el negocio.
- Cuando una regla del negocio cambia, hay un único lugar en el código donde cambiarla.

5. Errores comunes al aplicar DDD

- Crear **entidades anémicas**: clases con solo propiedades públicas y sin ningún comportamiento ni validación.
- Poner lógica de negocio en los controladores o casos de uso en lugar del dominio.



-
- Usar los DTOs (objetos de transferencia de datos) directamente como si fueran entidades de dominio.
 - Nombrar las clases con términos técnicos (`TblClientes` , `ClientDTO`) en lugar del lenguaje del negocio.

Tema 8 — ¿Qué es un modelo de dominio?

1. ¿Qué es?

El **modelo de dominio** es la representación en código de los conceptos, reglas y comportamientos del negocio. No es un esquema de base de datos ni un conjunto de formularios: es una abstracción que captura *la esencia de lo que el sistema debe hacer*.

2. Analogía

El modelo de dominio es como el **plano de un arquitecto**: no es el edificio real, pero lo representa con precisión. Muestra qué existe, cómo se relaciona y qué reglas debe respetar. El esquema de tablas que genera EF Core en la base de datos es la "construcción física" de ese plano, pero el plano lo manda el dominio.

3. El modelo de dominio de CU-CLI-001

Agregado: Client

Entidad raíz: **Client** (Aggregate Root)

Value Objects que componen al cliente:

- **FullName** – nombre (obligatorio, sin caracteres especiales)
- **LastName** – apellido (obligatorio)
- **Document** – DNI/CUIT con formato válido
- **Email** – dirección de correo válida y única en el sistema
- **Phone** – número de teléfono con formato válido
- **Address** – dirección postal del cliente
- **BirthDate** – fecha de nacimiento (el cliente debe ser mayor de edad)

Reglas de negocio del agregado:

- El cliente debe ser mayor de 18 años al momento del registro.
- El documento debe tener un formato válido.
- El email debe tener formato válido y ser único en el sistema.
- Nombre y apellido son obligatorios y no pueden estar vacíos.

4. Vista anticipada — cómo se expresa el modelo en C#

A continuación se muestra la **firma del diseño final** de la entidad **Client**. No es el código completo (se construye paso a paso desde el Módulo 1), sino una vista anticipada para que el alumno pueda visualizar hacia dónde vamos:

```
1 // Entidad Client – Aggregate Root del módulo Customers
2 // Proyecto: CommerX.Domain / Customers / Entities / Client.cs
3 public sealed class Client
4 {
5     // private set: el valor solo puede asignarse desde dentro de esta clase
6     public Guid Id { get; private set; }
7     public FullName FullName { get; private set; } // Value Object: valida el nombre
8     public LastName LastName { get; private set; } // Value Object: valida el apellido
9     public Document Document { get; private set; } // Value Object: valida el documento
10    public Email Email { get; private set; } // Value Object: valida el email
11    public Phone Phone { get; private set; } // Value Object: valida el teléfono
12    public Address Address { get; private set; } // Value Object: valida la dirección
13    public BirthDate BirthDate { get; private set; } // Value Object: valida mayoría de edad
14
15    // Constructor privado: nadie puede hacer "new Client()" desde afuera
16    // La única forma de crear un Client es usando el método Create()
17    private Client() { }
18
19    // Create(): método estático que construye y valida el cliente a la vez
20    // Si algún dato es inválido, se lanza una excepción antes de crear el objeto
21    public static Client Create(
22        string fullName, string lastName, string document,
23        string email, string phone, string address, DateTime birthDate)
24    {
25        // cada Value Object valida su dato al crearse
26        // si algo es inválido, la excepción se lanza aquí (Fail Fast)
27        return new Client
28        {
29            Id = Guid.NewGuid(), // identificador único generado automáticamente
30            FullName = FullName.Create(fullName), // lanza excepción si el nombre es inválido
31            LastName = LastName.Create(lastName), // lanza excepción si el apellido es
32            inválido
33            Document = Document.Create(document), // lanza excepción si el documento es
34            inválido
35            Email = Email.Create(email), // lanza excepción si el email es inválido
36            Phone = Phone.Create(phone), // lanza excepción si el teléfono es inválido
37            Address = Address.Create(address), // lanza excepción si la dirección es inválida
38            BirthDate = BirthDate.Create(birthDate) // lanza excepción si es menor de edad
39        };
40    }
41 }
```

Puntos clave para recordar:

- **Constructor privado:** nadie puede hacer `new Client()` desde afuera. El objeto siempre nace por `Create()`.
- **Método Create():** único punto de entrada. Garantiza que el objeto *siempre* nazca en un estado válido.
- **private set:** una vez creado el cliente, sus propiedades no se pueden modificar desde afuera sin pasar por un método del propio cliente.



- **Value Objects:** cada uno valida su dato. El cliente no valida el email: lo delega a `Email.Create()`.
- **El dominio no conoce EF Core, HTTP ni ninguna tecnología externa.** Es puro C#.

Actividades prácticas — Módulo 0

Las siguientes actividades no requieren escribir código todavía. Su objetivo es construir el modelo mental correcto antes de avanzar al Módulo 1.

Actividad 1 — Análisis del caso de uso CU-CLI-001

Consigna:

Leer el siguiente enunciado y responder las preguntas de reflexión:

"El sistema CommerX permite registrar nuevos clientes. Para registrar un cliente se requiere: nombre, apellido, documento, email, teléfono, dirección y fecha de nacimiento. El cliente debe ser mayor de 18 años. No pueden existir dos clientes con el mismo email ni con el mismo documento."

Preguntas de reflexión:

1. ¿Qué datos requieren validación de *formato* (ej: que el email tenga @)? ¿Cuáles requieren validación de *negocio* (ej: que sea mayor de edad)?
2. ¿Dónde debería vivir la regla "mayor de 18 años"? ¿En la base de datos, en el controlador o en el dominio? Justificar.
3. Si esa regla cambiara a "mayor de 21 años", ¿en cuántos lugares habría que modificarla si no hay arquitectura? ¿Y con Clean Architecture?
4. ¿Qué diferencia hay entre validar que un email *tenga formato correcto* y validar que un email *sea único* en el sistema? ¿Puede el dominio verificar la unicidad solo? ¿Por qué?

Actividad 2 — Identificar dependencias

Consigna:

Analizar las siguientes relaciones entre clases e indicar si la dependencia es correcta o incorrecta según Clean Architecture. Justificar cada respuesta:

Relación	¿Correcta o incorrecta? ¿Por qué?
<code>CreateClientUseCase</code> usa <code>IClientRepository</code>	
<code>Client</code> (Domain) usa <code>DbContext</code> de EF Core	
<code>ClientRepository</code> implementa <code>IClientRepository</code>	
<code>ClientsController</code> usa <code>CreateClientUseCase</code>	

Email (Value Object) usa
HttpClient para validar

Actividad 3 — Clasificar por capa

Consigna:

Clasificar cada elemento en su capa correcta de Clean Architecture:

Elemento	Capa: Domain / Application / Infrastructure / Presentation
Clase Client con sus reglas de negocio	
Clase CreateClientRequest (datos del formulario)	
Clase ClientRepository con EF Core	
Interfaz IClientRepository (contrato)	
Clase CreateClientUseCase	
Controller ClientsController	
Value Object Email	
Excepción InvalidEmailException	

Actividad 4 — Detectar violaciones SOLID

Consigna:

Analizar el siguiente enunciado e identificar qué principios SOLID viola y por qué: "Tenemos una clase llamada **ClientManager** que hace todo: valida el formato del email, verifica que el documento no esté repetido en la base de datos, guarda el cliente, envía un email de bienvenida y registra un log de auditoría."

Preguntas:

1. ¿Cuántas responsabilidades distintas tiene **ClientManager**? ¿Qué principio viola esto?

2. Si se quiere agregar envío por WhatsApp además del email de bienvenida, ¿hay que modificar `ClientManager`? ¿Qué principio viola esto?
3. ¿Cómo redesignarías el sistema repartiendo las responsabilidades? Escribir al menos los nombres de las clases que crearías.

Actividad 5 — Diseño previo al código

Consigna:

Sin escribir código, responder por escrito:

1. ¿Qué proyectos C# crearías en la solución CommerX y por qué uno por cada capa?
2. ¿Qué subcarpetas necesita el proyecto `CommerX.Domain`?
3. ¿Qué archivos vivirían en `CommerX.Domain/Customers/ValueObjects/`?
4. ¿Por qué `CommerX.Domain` no debe referenciar a `CommerX.Infrastructure`? Explicar usando el concepto de dependencia visto en el Tema 3.
5. ¿Dónde viviría la interfaz `IClientRepository`? ¿En Domain, en Application o en Infrastructure? ¿Por qué?

✓ Al completar este módulo, el alumno tiene el modelo mental necesario para comenzar a construir el dominio en el **Módulo 1 — Entidades**.